COM 123: PROGRAMMING USING JAVA

Module 1: Introduction

· Identify the basic components of Java programs.
· Distinguish two types of Java constructs Application and applets.
· Differentiate between object declaration and object creation.
· Describe the process of creating and running Java programs.
· Apply main window and message Box classes.
· Apply graphic classes

Why Java?

Thousands of programmers are embracing Java as the programming language of choice and several hundred more will joining before the end of the decade. Why is this so? The basic reasons for these are highlighted below:

a. **Portability:** Java is a highly portable programming language because it is not designed for any specific hardware or software platform. Java programs once written are translated into an intermediate form called bytecode. The bytecode is then translated by the Java Virtual Machine (JVM) into the native object code of the processor that the program is been executed on.

b. **Memory Management:** Java is very conservative with memory; once a resource is no longer referenced the garbage collector is called to reclaim the resource.

c. **Extensibility:** The basic unit of Java programs is the class. Every program written in Java is a class that specifies the attributes and behaviors of objects of that class. Java APIs (Application Programmers Interface) contains a rich set reusable classes that is made available to the programmers.

d. **Secure:** Java is a very secure programming language. Java codes (applets) may not access the memory on the local computer that they are downloaded upon. Thus it provides a secure means of developing internet applications.

e. **Simple:** Java's feature makes it a concise programming language that is easy to learn and understand. It is a serious programming language that easily depicts the skill of the programmer.

f. **Robustness:** Java is a strongly typed programming language and encourages the development of error free applications.

Types of Java Programs

Java programs may be developed in three ways:

a. **Java Applications:** These are stand-alone applications such as word processors, inventory control systems etc.

b. **Java Applets:** These programs that are executed within a browser. They are executed on the client computer.

c. **Java Serverlets:** These are server side programs that are executed within a browser.

In this course we will limit ourselves to only the first two mentioned types of Java programs – applications and applets.

Introduction to Java Applications

As earlier described Java applications are stand alone programs that can be executed to solve specific problems. Before delving into the details of writing Java applications (and applets) we will consider the concept on which the language is based upon being: Object Oriented.

Programming (OOP).

Object Oriented Programming is a methodology which has greatly revolutionized how programs are designed and developed as the complexities involved in programming are increasing. The following are the basic principles of OOP.

a. **Encapsulation:** Encapsulation is a methodology that binds together data and the codes that it manipulates thus keeping it safe from external interference and misuse. An object oriented program contains codes that may have private members that are directly accessible to only the members of that program.

b. **Polymorphism:** Polymorphism is a concept whereby a particular "thing" may be employed in many forms and the exact implementation is determined by the specific nature of the situation (or problem).

c. **Inheritance:** Inheritance is the process of building new classes based on existing classes. The new class inherits the properties and attributes of the existing class. Object oriented programs models real world concepts of inheritance. For example children inherit attributes and behaviors from their parents. The attributes such as color of eyes, complexion, facial features etc represent the fields in an java. Behaviors such as being a good dancer, having a good sense of humor etc represent the methods. The child may have other attributes and behaviors that differentiate them from the parents.

Components of a Java Application Program

Every Java application program comprises of a class declaration header, fields (instance variables – which is optional), the main method and several other methods as required for solving the problem. The methods and fields are members of the class.

In order to explore these components, let us write our first Java program.

```
/*
* HelloWorld.java
* Displays Hello world!!! to the output window
*
*/
public class HelloWorld // class definition header
{
public static void main( String[] args )
{
System.out.println("Hello World!!! " ); // print text
} // end method main
} // end class HelloWorld
```

The above program is a simple yet complete program containing the basic features of all Java application programs. We will consider each of these features and explain them accordingly.

The first few lines of the program are comments.

```
/*
* HelloWorld.java
* Displays Hello world!!! to the output window
*
*/
```

The comments are enclosed between the /* */ symbols.

Comments are used for documenting a program, that is, for passing across vital information concerning the program – such as the logic being applied, name of the program and any other relevant information etc. Comments are not executed by the computer.

Comments may also be created by using the // symbols either at the beginning of a line:

// This is a comment

Or on the same line after with an executable statement. To do this the comment must be written after the executable statement and not before else the program statement will be ignored by the computer:

System.out.println("Hello World!!! "); **// in-line comment.**

This type of comment is termed as an in-line comment.

The rest of the program is the class declaration, starting with the class definition header:

public class HelloWorld, followed by a pair of opening and closing curly brackets.

{

}

The class definition header class definition header starts with the access modifier **public** followed by the keyword **class** then the name of the class **HelloWorld**. The access modifier tells the Java compiler that the class can be accessed outside the program file that it is declared in. The keyword **class** tells Java that we want to define a class using the name HelloWorld.

Note: The file containing this class must be saved using the name HelloWorld.java. The name of the file and the class name must be the same both in capitalization and sequence. Java is very case sensitive thus HelloWorld is different from helloworld and also different from HELLOWORLD.

The next part of the program is the declaration of the main method. Methods are used for carrying out the desired tasks in a Java program.

The code:

public static void main(String[] args) {

}

is the main method definition header. It starts with the access modifier public, followed by the keyword static which implies that the method **main( )** may be called before an object of the class has been created. The keyword void implies that the method will not return any value on completion of its task. These keywords public, static, and void should always be placed in the sequenced shown.

Any information that you need to pass to a method is received by variables specified within the set of parentheses that follow the name of the method. These variables are called parameters. If no parameters are required for a given method, you still need to include the empty parentheses.

In **main( )** there is only one parameter, **String[] args**, which declares a parameter named **args**. This is an array of objects of type **String**. (*Arrays* are collections of similar objects.) Objects of type **String** store sequences of characters. In this case, **args** receives any command-line arguments present when the program is executed. Note that the parameters could have been written as **String args[]**. This is perfectly correct.

The instructions (statements) enclosed within the curly braces will be executed once the main method is run. The above program contains the instruction that tells Java to display the output “Hello World!!!” followed by a carriage return. This instruction is:

System.out.println(“Hello World!!!”); //print text

This line outputs the string "Hello World" followed by a new line on the screen. Output is actually accomplished by the built-in **println( )** method. In this case, **println( )** displays the string which is passed to it. As you will see, **println( )** can be used to display other types of information, too. The line begins with **System.out**. While too complicated to explain in detail at this time, briefly, **System** is a predefined class that provides access to the system, and **out** is the output stream that is connected to the console. Thus, **System.out** is an object that encapsulates console output. The fact that Java uses an object to define console output is further evidence of its object-oriented nature.

As you have probably guessed, console output (and input) is not used frequently in real-world Java programs and applets. Since most modern computing environments are windowed and graphical in nature, console I/O is used mostly for simple utility programs and for demonstration programs. Later you will learn other ways to generate output using Java, but for now, we will continue to use the console I/O methods. Notice that the **println( )** statement ends with a semicolon. All statements in Java end with a semicolon. The reason that the other lines in the program do not end in a semicolon is that they are not, technical statements.

The first closing brace -}- in the program ends **main( )**, and the last } ends the **HelloWorld** class definition; it is a good practice to place a comment after the closing curly brace. The opening and close brace are referred to as a block of code.

One last point: Java is case sensitive. Forgetting this can cause you serious problems. For example, if you accidentally type **Main** instead of **main**, or **PrintLn** instead of **println**, the preceding program will be incorrect. Furthermore, although the Java compiler *will* compile classes that do not contain a **main( )** method, it has no way to execute them. So, if you had mistyped **main**, the compiler would still compile your program. However, the Java interpreter would report an error because it would be unable to find the **main( )** method.

In the above program some lines where left blank, this was done in order to make the program readable. Furthermore, tabs (indentation) were used to within the body of a class or methods as appropriate to spate characters and symbols. The blank spaces, tabs, and newline characters are referred to as white spaces.

Compilation and Execution of Java Programs

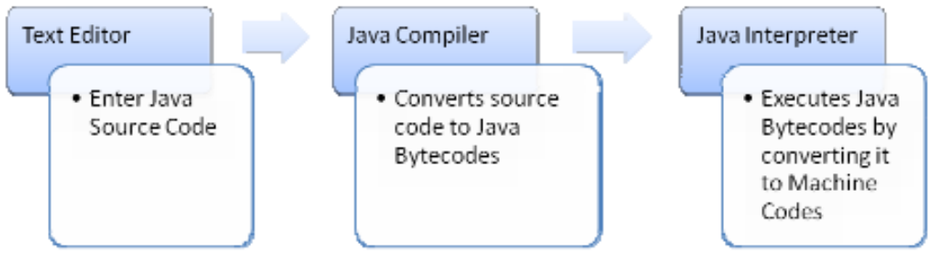
As earlier mentioned in this text we will create only two types of Java programs – applications and applets. In the next few paragraphs the steps for editing, compiling and executing a Java programs. The procedures for Java application and Java applets are basically the same. The major difference is that Java applets are executed within a browser.

The basic steps for compiling and executing a Java program are:

a. Enter the source code using a text editor. He file must be saved using the file extension .java.

b. Use the Java compiler to convert the source code to its bytecode equivalent. The byte code will be saved in a file having the same name as the program file with an extension .class. To compile our HelloWorld.java program, type the following instructions at the Windows command prompt (c:\>): javac HelloWorld.java The bytecodes (.class file) will be created only if there are no compilation errors.

c. Finally use the Java interpreter to execute the application, to do this at the Windows command prompt (c:\>) type: java HelloWorld. (You need not type the .class extension).



Note: Other programs, called *Integrated Development Environments* (IDEs), have been created to support the development of Java programs. IDEs combine an editor, compiler, and other Java support tools into a single application. The specific tools you will use to develop your programs depend on your environment. Examples of IDEs include NetBeans, Eclipse, BlueJ etc.

Lecture 2:

1. Using Simple Graphical User Interface

2. Apply Graphical Classes

Using Simple Graphical Interface

This week we will employ simple graphical classes – JOptionPane to repeat the same programs which we implemented in week one. In the program presented in week one the output was presented to the windows command prompt.

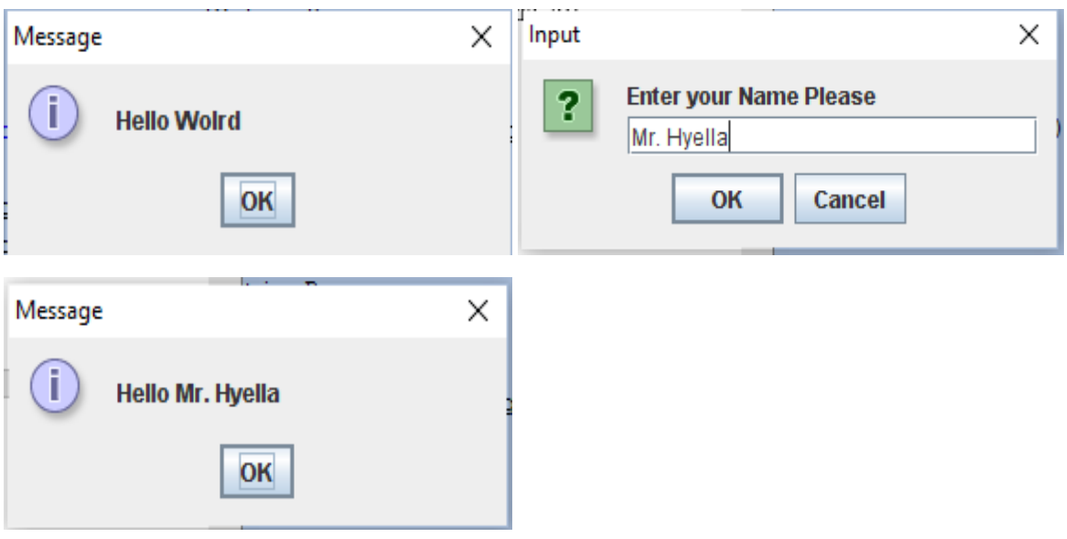
The JOptionPane class (javax.swing package) enables the user to use its static methods showInputDialog and showMessageDialog to accept data and display information graphically. The HelloWorldGUI.java which implements JOptionPane static methods for displaying hello

world to the user is presented below:

```
1 /*
2 * HelloWolrdGUI.java
3 *
4 */
5
6
7 import javax.swing.JOptionPane;
8
9 public class HelloWorldGUI {
10 public static void main(String[] args) {
11 String msg = "Hello Wolrd";
12 String ans ="";
13
14 JOptionPane.showMessageDialog(null, msg );
15
16 // accept the user's name
17 ans = JOptionPane.showInputDialog( null, "Enter your Name Please"
);
18
19 // say hello to the user
20 JOptionPane.showMessageDialog(null, "Hello " + ans );
21
22 } // end method main
23
24 } // end of class HelloWorldGUI
```

Line 7 we imported he JOptionPane class so that the JVM will ensure that we use it properly. The class definition header is presented in line 9. This is followed by the main method header which must be written this way it is presented in line 10. Two string variables are used, one for displaying output – msg – and the other for input –ans-. The Graphical message is displayed with "Hello World" and a command button labeled ok shown. The user is requested to enter his/her name (line 17) and a hello message with the name enter is displayed –(line 20).

The outputs of the program are presented below.



Module 2:

DATA TYPES IN JAVA

A **data type** defines a set of values and the operations that can be defined on those values. Data types in Java can be divided into two groups:

a. Primitive Data Types

b. Reference Data Types (or Non-Primitives)

Data types are especially important in Java because it is a strongly typed language. This means that all operations are type checked by the compiler for type compatibility. Illegal operations will not be compiled. Thus, strong type checking helps prevent errors and enhances reliability. To enable strong type checking, all variables, expressions, and values have a type. There is no concept of a "type-less" variable, for example. Furthermore, the type of a value determines what operations are allowed on it. An operation allowed on one type might not be allowed on another.

Primitive Data Types

The term primitive is used here to indicate that these types are not objects in an object-oriented sense, but rather, normal binary values. These primitive types are not objects because of efficiency concerns. All of Java's other data types are constructed from these primitive types.

There are *eight primitive data types in Java*: four subsets of integers, two subsets of floating point numbers, a character data type, and a boolean data type. Everything else is represented using objects. Let's examine these eight primitive data types in some detail.

Integers and Floating Points

Java has two basic kinds of numeric values: integers, which have no fractional part, and floating points, which do. There are four integer data types (byte, short, int, and long) and two floating point data types (float and double). All of the numeric types differ by the amount of memory space used to store a value of that type, which determines the range of values that can be represented. The size of each data type is the same for all hardware platforms. All numeric types are signed, meaning that both positive and negative values can be stored in them. The table below summarizes the numeric primitive types.

Type	Storage	Minimum Value	Maximum Value
byte	8 bits	-128	127
short	16 bits	–32,768	32,767
int	32 bits	–2,147,483,648	2,147,483,647
long	64 bits	–9,223,372,036,854,775,808	9,223,372,036,854,775,807
float	32 bits	Approximately –3.4E+38 with 7 significant digits	Approximately 3.4E+38 with 7 significant digits
double	64 bits	Approximately –1.7E+308 with 15 significant digits	Approximately 1.7E+308 with 15 significant digits

A *literal* is an explicit data value used in a program. The various numbers used in programs such as Facts and Addition and Piano Keys are all *integer literals.*

Java assumes all integer literals are of type *int*, unless an L or l is appended to the end of the value to indicate that it should be considered a literal of type *long*, such as 45L.

Likewise, Java assumes that all *floating point literals* are of type *double*. If we need to treat a floating point literal as a *float*, we append an F or f to the end of the value, as in 2.718F or 23.45f. Numeric literals of type *double* can be followed by a D or d if desired.

The following are examples of numeric variable declarations in Java:

int marks = 100;

byte smallNo1, smallNo2;

long totalStars = 86827263927L;

float ratio = 0.2363F;

double mega = 453.523311903;

Arithmetic Operators

Arithmetic operators are special symbols for carrying out calculations. These operators enable programmers to write arithmetic expressions. An ***expression*** is an algebraic like term that evaluates to a value; it comprises of one or more *operands* (values) joined together by one or more *operators*. Below is a summary of Java arithmetic operators in their *order of precedence*, that is, the order in which the arithmetic expression are evaluated.

Order of Precedence	Operator	Symbol	Algebraic Expression	Java Expression	Association
First	Multiplication	*	a x c	a * c	Left to Right
	Division	/	x/y or x ÷ y or $\frac{x}{y}$	x/y	Left to Right
	Modulus or Remainder	%	w mod 3	w % 3	Left to Right
Second	Addition	+	d+ p	d + p	Right to Left
	Subtraction	-	j – 2	j - 2	Right to Left

Precedence of Arithmetic Operators

The order in which arithmetic operators are applied on data values (operand) is termed rules of operator precedence. These rules are similar to that of algebra. They enable Java to evaluate arithmetic expressions consistently and correctly.

The rules can be summarized thus:

a. Multiplication, division and modulus are applied first. Arithmetic expressions with several of these operators are evaluated from the left to the right.

b. Addition and subtraction are applied next. In the situation that an expression contains several of these operators they are evaluated from right to left.

The order in which the expressions are evaluated is referred to as their association. Now let us consider some examples in the light of the rules of operator precedence; we will list both the algebraic expression and the equivalent java expression.

Algebra : $k = \frac{x+y+z}{3}$

Java: k = (x + y + z)/3;

This expression calculates the average of three values. The parentheses is required so that the values represented by x, y and z will be added and the result divided by three. If the parentheses is omitted only z will be divided by three because division has a higher precedence over addition.

Algebra: y = mx + c

Java: y = m * x + c;

In this case the parentheses is not required because multiplication has higher precedence over addition.

Algebra: z = pr % q + $\frac{w}{x}$ – y

Java: z = p * r % q + w/x – y;

Note: the order of precedence may be overwritten by using parentheses, that is to say if we desire addition before multiplication or division for example we can include the that part of the expression in parentheses. In the above expression, if x - y is written as (x – y), then the value represented by the y will be subtracted from that of x then the result will be divided by w.

Exercises

Show the order of execution the following arithmetic expressions and write their Java equivalents:

i. $a = \frac{4}{2} - (4 + 1x)$

ii. $c = w - 2d^4 - \frac{k}{f+2}$

iii. $r = 2c + wxy^2$

iv. $y = mx + 3b$

v. $T = 4r + d \ mod \ 4$

Example: To demonstrate arithmetic operations by creating a program to perform simple arithmetic.

```
/**
 *
 * @author Mr. Hyella
 */
public class ArithmeticTest {
   public static void main(String[] args) {
                // TODO code application logic here
            short x = 6;
            int y = 4;
            float a = 12.5f;
            float b = 7f;
            System.out.println("x is " + x + ", y is " + y);
            System.out.println("x + y = " + (x + y));
            System.out.println("x - y = " + (x - y));
            System.out.println("x / y = " + (x / y));
            System.out.println("x % y = " + (x % y));
            System.out.println("a is " + a + ", b is " + b);
            System.out.println("a / b = " + (a / b));
  } // end of method main
} // end of class
```

Output

```
x is 6, y is 4
x + y = 10
x - y = 2
x / y = 1
x % y = 2
a is 12.5, b is 7.0
a / b = 1.7857143
```

Reference (Non-primitive Data Types)

Reference or non-primitive type data is used to represent objects. An object is defined by a *class,* which can be thought of as the data type of the object. The operations that can be performed on the object are defined by the methods in the class. The attributes or qualities of the objects of a class are defined by the fields – which in essence are primitive type data values.

Every object belongs to a class and can be referenced using *identifiers*.

An **identifier** is a name which is used to identify programming elements such as memory location, names of classes, Java statements and so on. The names used for identifying memory locations are commonly referred to as memory variables or variables for short.

Variable names are created by the programmer for representing values to be stored in the computer memory. Each memory location is associated with a type, a value, and a name. *Primitive data* such as int (integer) can hold a single value and that value must correspond to the data type specified by the programmer. *Reference data types* on the other hand contain not the objects in memory but the addresses of where the objects (their method and fields etc) are stored in memory. Examples of reference data types include arrays, strings and objects of any class declared by the programmer.

The rules for creating variables in Java.

a. Variables names can start with and alphabet (a-z / A-Z) and the remaining characters may be, an underscore (_) or a dollar sign, or a number for example sum, counter, firstName, bar2x amount_Paid are all valid variable names. 9x, 0value are invalid.
b. Embedded blank spaces may not be included in variable names though an underscore may be used to join variable names that comprises of compound words. Example x 10 is not valid, it could be written as x_10 or x10.
c. Reserved words (words defined for specific use in Java) may not be employed as variables names. Example loop, do, for, while, switch are reserved.

d. Special symbols such as arithmetic operators, comma (,), ?, /, ! are not allowed. e. Variable name may be of any length.

Java is a case sensitive programming language thus the programmer must be very careful and consistent when giving and using variable names. Java distinguishes between upper case (capital) letters and lower case letters (small letters) hence 'a' is different from 'A' as far as Java is concerned.

Exercises: Study the variable names below and indicate whether they are valid or not. If invalid give reasons.

1. &maxium _____
2. X _____
3. Absolute temperature _____
4. Money _____
5. 30yearold _____
6. initVolume _____

Variable Declaration

Variable may represent values that are expected to change or not during the execution of a computer program. When declaring variable names the scope (visibility) of variables from other part of the program may be specified, the type of data that should be stored in the area of memory.
Variable names may also represent either primitive data or reference data.

The general form for creating or declaring variable is presented below.
accessModifier dataType variableList
Where:
accessModifier determines the visibility of the variable to other part of the program e.g. public or private.
dataType represents either primitive (int, char, float) or reference type data (array, String).
variableList is one or more valid variables separated using commas.
Examples:

a. private int x, y, z;
   In this example the access modifier is private, the data type is int and the variables that are permitted to hold integer values are the identifiers x, y and z. similar pattern is applied in other examples below.
b. private float balance, initTemperature;
c. private boolean alreadyPaid;
d. public long population;

Alternatively the initial values to be stored in the memory may be specified when declaring the variables. The general form for declaring and initializing the variables is presented below:
accessModifier dataType variable1= value1, variable2 = value2, … , variable = valuen;
We will illustrate with examples.

```
private int x = 10;
private int a = 0, b= 0, c = 0;
public double amountLoaned = 100;
```

Note: we declaring variables in a method (e.g. main method) do not include the access modifiers because all variables declared in methods are implicitly private hence localized to that method. The examples given above can be used to create *instance variables.*

Now let us write a program to put together all that we have learnt. The program listing below demonstrates how to create primitive variables (int) and non-primitive variable (of type Scanner). It demonstrates how to write arithmetic expressions, input and output data from the user.

```
/**
 * AddTwoNo.java
* Add any two integer numbers
 * @author Mr. Hyella
 */
```

```
import java.util.Scanner;
public class AddTwoNo {
        /**
          * @param args the command line arguments
          */
        public static void main(String[] args) {
                // declare primitive variables
                int firstNumber = 10;
                int secondNumber = 20;
                int sum = 0;
                sum = firstNumber + secondNumber;
                System.out.println( "Sum of " + firstNumber + " and " +
                secondNumber + " is " + sum );
                // declare reference variable of type Scanner
                Scanner input = new Scanner( System.in );
                // Accept values from the user
                System.out.println( "Enter first integer number please ");
                firstNumber = input.nextInt();
                System.out.println( "Enter second integer number please ");
                secondNumber = input.nextInt();
                // calculate sum and display result
                sum = firstNumber + secondNumber;
                System.out.println( "Sum of " + firstNumber + " and " +
                secondNumber + " is " + sum );
        } // end of method main
} // end of class AddTwoNos
```

Note:

import java.util.Scanner;

This statement instructs Java to include the Scanner as part of our program so that it will be able to ensure that we use the elements of this class properly. A collection of classes are grouped together in Java for ease of usage and to facilitate software reuse. A collection of classes grouped together are referred to as a package. The Scanner class is a member of the java.util package. By importing classed and using them in our classes makes Java a robust programming language.

Using Graphical User Interfaces

The entire program we have written so far has inputted or displayed prompts to the user via the output window. In most real world programs, this will not be the case. Most modern applications display messages to the user using windows – dialog boxes – and use same to accept data from the user. In our next example, we will improve on the addition program by using dialog boxes.

```
/**
 * AddTwoNoDialog.java
* Add any two integer numbers
 * @author Mr. Hyella
 */
import javax.swing.JOptionPane;
public class AddTwoNoDialog {
        public static void main(String[] args) {
                // declare primitive variables
                int firstNumber = 10;
                int secondNumber = 20;
                int sum = 0;
                String input; // for accepting input from the user
                String output; // for displaying output
                // Accept values from the user
                input = JOptionPane.showInputDialog( null, "Enter first integer number",
                "Adding Integers", JOptionPane.QUESTION_MESSAGE );
                firstNumber = Integer.parseInt( input );
```

```
                input = JOptionPane.showInputDialog(null, "Enter second integer number",
                "Adding Integers", JOptionPane.QUESTION_MESSAGE );
                secondNumber = Integer.parseInt( input );
                // calculate sum and display result
                sum = firstNumber + secondNumber;
                // build output string
                output = "Sum of " + firstNumber + " and " + secondNumber + " is "
                + sum;
                // display output
                JOptionPane.showMessageDialog( null, output, "Adding two integers",
                JOptionPane.INFORMATION_MESSAGE );
} // end of method main
} // end of class AddTwoNos
```

Note:

In this program, we imported the JOptionPane class (package javax.swing). JoptionPane contains several *static methods* and *static fields.*

In the program we imported the JOptionPane class (import javax.swing.JOptionPane).

This will ensure Java loads the class, and this class enables us to create objects that displays dialog boxes for both input and output.

Module 3:

Program Development Stages

Programming in any programming language is not a task that should be trivialized as mere entry of code into the computer. In order to develop robust and efficient applications the developer/programmer must follow certain steps. Most beginning programmers simply start keying in code, for simple applications this may be ok, for most real-life applications that may include hundreds of classes and codes spanning thousands of lines such an approach to programming is as good as a Mission Impossible.

In this chapter we will introduce the basic steps that are to be employed when developing a Java program. Programming basically involves problem solving – that is we write programs to efficiently and effectively meet the needs of the users.

Problem solving

The purpose of writing a program is to solve a problem. Problem solving, in general, consists of multiple steps:

a. Understanding the problem.
b. Breaking the problem into manageable pieces.
c. Designing a solution.
d. Considering alternatives to the solution and refining the solution.
e. Implementing the solution.
f. Testing the solution and fixing any problems that exist.

The first step, understanding the problem, may sound obvious, but a lack of attention to this step has been the cause of many misguided efforts. If we attempt to solve a problem we don't completely understand, we often end up solving the wrong problem or at least going off on improper tangents. We must understand the needs of the people who will use the solution.

After we thoroughly understand the problem, we then break the problem into manageable pieces and design a solution. These steps go hand in hand. A solution to any problem can rarely be expressed as one big activity. Instead, it is a series of small cooperating tasks that interact to perform a larger task. When developing software, we don't write one big program. We design separate pieces that are responsible for certain parts of the solution, subsequently integrating them with the other parts.

The act of designing the program should be more interesting and creative than the process of implementing the design in a particular programming language. Finally, we test our solution to find any errors that exist so that we can fix them and improve the quality of the software. Testing efforts attempt to verify that the program correctly represents the design, which
in turn provides a solution to the problem.

Example: Finding an average of three numbers.

Solution:

What are the major processes involved in calculating the sum and average of any three numbers? These could be summarized as follows:

a. First we must be able to accept any three numbers from the user.
b. Calculate the average.
c. Display average.

Considering step 'a' we do not need to break it down. Let us implement this in our program using dialog boxes.

Then step 'b' - calculate the average – how do we calculate average of three integer numbers?
This can be broken down into two steps that is:

i. Add up the three numbers to calculate their sum.

```
Sum = firstNumber + secondNumber + thirdNumber
```

ii. Divide the sum by three to calculate the average.

```
Average = sum / 3
```

The resulting value – average – may be an integer value or a floating-point value. This being the case we would declare average as double.

Finally, we consider the step c, that is, displaying the result. We will display the numbers added and then the average using dialog boxes also.

The entire processing steps (algorithm) are rewritten as:

a. First we must be able to accept any three numbers from the user.
b. Calculate the average.

i. Add up the three numbers to calculate their sum.
ii. Divide the sum by three to calculate the average.

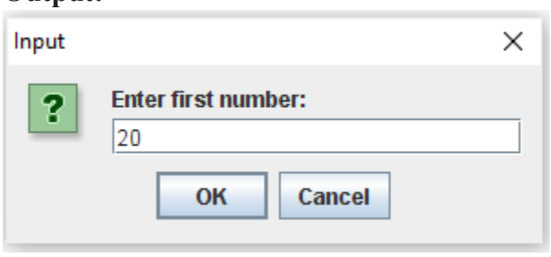
c. Display average

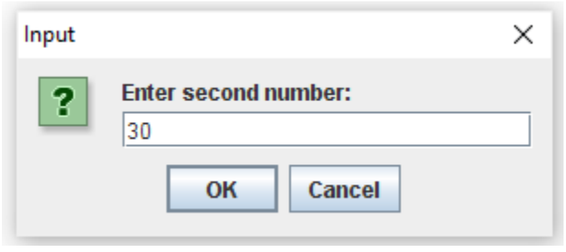
Now we proceed to write the Java program, correct any errors and test with sample data. The source code for the program is presented below:

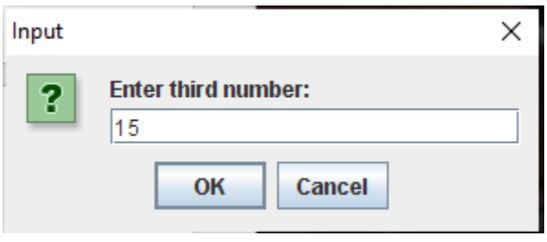
Program code:

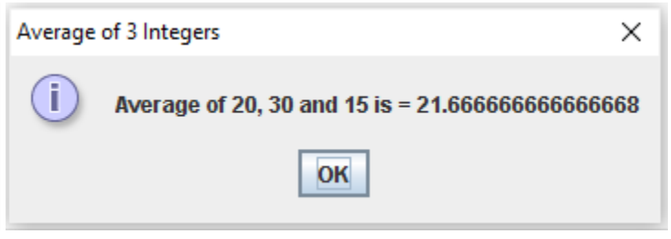
```
/**
 *
 * @author Mr. Simon
 * Program to calculate the Average Three Integers
 */
import javax.swing.JOptionPane;
public class AverageThreeIntegers {
    /**
     * @param args the command line arguments
     */
   public static void main(String[] args) {
        // TODO code application logic here
        int firstNumber; // first integer number
        int secondNumber; // second integer number
        int thirdNumber; // third integer number
        int sum; // sum of the three numbers
        double average; // average of the three numbers
        String input; // input values
        String result; // output generating string
        // Accept inteher numbers from he user
        input = JOptionPane.showInputDialog( null, "Enter first number: " );
        firstNumber = Integer.parseInt( input ); // wrap input to integer
        input = JOptionPane.showInputDialog( null, "Enter second number: " );
        secondNumber = Integer.parseInt( input ); // wrap input to integer
        input = JOptionPane.showInputDialog( null, "Enter third number: " );
        thirdNumber = Integer.parseInt( input ); // wrap input to integer
        // Calculate sum
        sum = firstNumber + secondNumber + thirdNumber;
        // Calculate average
        average = sum/3.0;
        // Build output string and display output
        result = "Average of " + firstNumber + ", " + secondNumber + " and " +
        thirdNumber + " is = " + average;
        JOptionPane.showMessageDialog( null, result, "Average of 3 Integers",
        JOptionPane.INFORMATION_MESSAGE );
    }
}
```

Output:









Module 4:

Insatiable Classes

Insatiable classes can permit users to create instances – objects of the class. Each object of the class will have its own set of variables – instance variables that represent the current state of the object and methods that defines the task that object can perform. Technically each method should perform only a single task, and it is permitted to call other methods to assist it.

Instance variables are declared outside any method inside the class and usually immediately after the class definition header. The access modifier is used to ensure that each object maintains it own set of variables and thus not visible to any other object of the class.

Declaring a Class with a Method and Instantiating an Object of a Class

We begin with an example that consists of classes Circle and CircleTest in the code below.

Class Circle (declared in file Circle.java) will be used to define the properties of a Circle object and tasks which each object of the class will be able to perform – in this case calculate the area of a Circle instance.

Class CircleTest (declared in file CircleTest.java) is an application class in which the main method will use class Circle. Each class declaration that begins with keyword public must be stored in a file that has the same name as the class and ends with the .java file-name extension. Thus, classes Circle and CircleTest must be declared in separate files, because each class is declared public.

Code:

```
1 /*
2 * Circle.Java
3 * Create a Circle Object and Calculate the Area of the Circle
4 */
5
6 public class Circle {
7 // Create Instance Variables
8 private double radius;
9
10 public Circle() {
11 radius = 0;
12 }
13
14 // User defined Constructor for Creating Circle Object with Specified
15 // radius - r
16 public Circle(double r){
17 setRadius( r );
18 }
19
20 // setRadius initializes radius and ensuees that it is always in a
21 // consistent state
22 public void setRadius(double r){
23 radius = r;
24
25 } // end method setRadius
26
27 // getRadius returns radius to clients of Circle class
28 public double getRadius(){
29 return radius;
30
31 } // end method getRadius
32
33 // Calculates the area of the circle
34 public double calcArea(){
35 return Math.PI * Math.pow(radius, 2);
36
37 } // end method calcArea
38 } // end class Circle
```

Class **Circle**

The Circle class declaration (Fig. 3.1) contains a two constructor methods, a no argument constructor Circle()method (lines 10-12) that initializes the radius of a Circle object to its default value of zero (0. The second constructor method is a programmer declared constructor Circle(double r) method (lines 16-18) that that initializes a Circle object using the values specified by the user.

The methods setRadius()and setRadius() are public service methods that enables the instance variable **radius** (declared in line 8) to be visible to other classes. The class declaration also contains calcArea()that calculates the area of the circle.
So far, each class we declared had one method named main (a special method that is always called automatically by the Java Virtual Machine (JVM) when you execute an application). Most methods do not get called automatically. As you will soon see, you must call method calcArea
to tell it to perform its task.

The method declaration begins with keyword public to indicate that the method is "available to the public" that is, it can be called from outside the class declaration's body by methods of other classes. Keyword double indicates that this method will perform a task and will return (i.e., give back) a value of type double to its **calling method** when it completes its task. Method setRadius on the other hand does not return any value to its calling method thus the keyword void.

The name of the method, calcArea, follows the return type. By convention, method names begin with a lowercase first letter and all subsequent words in the name begin with a capital letter. The parentheses after the method name indicate that this is a method. An empty set of parentheses, as shown in line 34, indicates that this method does not require additional information to perform its task. Line 7 is commonly referred to as the **method header**. Every method's body is delimited by left and right braces ({ and }), as in lines 34 and 37.

The body of a method contains statement(s) that perform the method's task. In this case, the method contains one statement (line 35) that calculates the area of a cicle. After this statement executes, the method has completed its task.
Next, we'd like to use class Circle in an application. As earlier stated, method main begins the execution of every application. A class that contains method main is a Java application. Such a class is special because the JVM can use main to begin execution. Class Circle is not an application because it does not contain main. Therefore, if you try to execute Circle by typing java Circle in the command window, you will get the error message:
Exception in thread "main" java.lang.NoSuchMethodError: main

Class **CircleTest**
The CircleTest class declaration (code below) contains the main method that will control our application's execution. Any class that contains main declared as shown on line 7 can be used to execute an application. This class declaration begins at line 4 and ends at line 16. The class contains only a main method, which is typical of many classes that begin an application's execution.

```
1 /*
2 * CircleTest.java
3 *
4 */
5
6 package MyTrig;
7
8 import javax.swing.JOptionPane;
9
10 public class CircleTest
11 {
12
13 public static void main(String[] args)
14 {
15 // Declare local variables
16 String input;
17 String output;
18
19 double newRadius = 0;
20
21 // Create Circle object using the no-argument constructor
22 Circle circle1 = new Circle();
23
24 // create another Circle instance class using the Programmer
25 // declared constructor
26 Circle circle2 = new Circle( 5 ); // circle has a radius of 5
27
28 // display state of Circle objects
```

```
29 output = "Radius = " + circle1.getRadius() +
30 "\nArea = " + circle1.calcArea();
31 JOptionPane.showMessageDialog( null, output, "Circle1",
32 JOptionPane.INFORMATION_MESSAGE );
33
34 output = "Radius = " + circle2.getRadius() +
35 "\nArea = " + circle2.calcArea();
36 JOptionPane.showMessageDialog( null, output, "Circle2",
37 JOptionPane.INFORMATION_MESSAGE );
38
39 // Allow user to alter the state of circle1 object
40 input = JOptionPane.showInputDialog( null, "Enter Radius: " );
41 newRadius = Double.parseDouble( input );
42
43 circle1.setRadius( newRadius ); // reset radius variable
44
45 // display current radius and area of circle1
46 output = "Radius = " + circle2.getRadius() +
47 "\n Area = " + circle2.calcArea();
48 JOptionPane.showMessageDialog( null, output, "Circle2",
49 JOptionPane.INFORMATION_MESSAGE );
50
51 } // end method main
52
53 } // end class CircleTest
```

Figure 5.2

Lines 13-51 declare method main. Recall that the main header must appear as shown in line 13; otherwise, the application will not execute. A key part of enabling the JVM to locate and call method main to begin the application's execution is the static keyword (line 13), which indicates that main is a static method. A static method is special because it can be called without first creating an object of the class in which the method is declared.

In this application, we'd like to call class Circle's calcArea method to calculate the area of any Circle object. Typically, you cannot call a method that belongs to another class until you create an object of that class, as shown in lines 30 and 47. We begin by declaring two variables circle1 and circle2. Note that the variable's type is Circle the class we declared.

Each new class you create becomes a new type in Java that can be used to declare variables and create objects. Programmers can declare new class types as needed; this is one reason why Java is known as an extensible language.

Variables circle1 and circle2 are initialized with the result of the **class instance creation expression** new Circle()and circle2(5) respectively. Keyword **new** creates a new object of the class specified to the right of the keyword (i.e., Circle). The parentheses to the right of the Circle are required. Those parentheses in combination with a class name represent a call to a constructor, which is similar to a method, but is used only at the time an object is created to initialize the object's data.

Just as we can use object System.out to call methods print, printf and println, we can now use Circle objects to call methods calcArea, setRadius and getRadius. Line 30 calls the method calcArea (declared at lines 34-37 of Fig. 5.1) using variable circle1 followed by a **dot separator** (.), the method name calcArea and an empty set of parentheses. This call causes the calcArea method to perform its task. In line 29, "circle1" indicates that main should use the Circle object that was created on line 22. Line 34 of the given code indicates that method calcArea has an empty parameter list that is, calcArea does not require additional information to perform its task. For this reason, the method call (line 29 of Fig. 5.2) specifies an empty set of parentheses after the method name to indicate that no arguments are being passed to method calcArea. When method calcArea completes its task, method main continues executing at line 51. This is the end of method main, so the program terminates. See Figures 5.1a-5.1d for sample run output.

Methods and Constructors a Deeper Look

As we earlier mentioned, methods are used for specifying the tasks that objects of the class can perform. Constructors are special methods that are used for initializing objects when they are created. In order to see the differences between the constructors and regular (non-constructor) methods, we have presented below the structure of a method:

```
accessModifer returnType methodName( parameterList )
{
        statements
        return statement
}
```

Where:
accessModifer (access modifier) specifies the visibility of the method to other classes – the access modifier may be specified public or private. Public specifies that the method is visible to other classes "that is visible to the public" while private is the exact opposite. Private methods cannot be accessed outside the class from which it is defined and thus it cannot be inherited.
returnType specifies the "nature" of the value the method gives back to its calling method (if any) when it completes its task. The return type could a primitive type value or reference type value as the case may be.
methodName (method name) is an identifier used to make reference to the method.
parameterList is an optional list of identifiers representing the values (arguments) to be passed into the method. The parameters –often referred to as formal parameters – are used by methods in carrying out of their tasks. They must have a type followed by an identifier. Several parameters must be separated using commas.
statements – instructions that enable the method to carry out its tasks.
return statement is used to return (send) data back to the calling method. When the return statement is implemented without a value following it, then the returnType must be specified as void.

Both regular methods and constructors are permitted to have parameters (formal parameters).
Note: values passed into methods and constructors are referred to as actual parameters which are copies of the actual data except when the data are of type reference.

Differences between Methods and Constructors
a. Constructors must have the same name as the class - both in capitalization and in sequence - in which it is declared.
b. Constructors do not have a return type in its method definition header.
c. The return statement is not required.
d. Constructors are only executed when an object is created, they may not be invoked if the state of the object alters. In such a situation that the state of the object changes, public service methods will be required to assist in reflecting the change.
e. Constructors may not be declared as static as they are involved in the creation and initialization of the objects themselves.

Module 5:
Introduction to Applets

There are two kinds of Java programs: Java applets and Java applications. A Java *applet* is a Java program that is intended to be embedded into an HTML document, transported across a network, and executed using a Web browser.

A Java *application* is a stand-alone program that can be executed using the Java interpreter. All programs presented thus far have been Java applications The Web enables users to send and receive various types of media, such as text, graphics, and sound, using a point-and-click interface that is extremely convenient and easy to use. A Java applet was the first kind of executable program that could be retrieved using Web software. Java applets are considered just another type of media that can be exchanged across the Web.

A class that defines an applet extends the JApplet class, as indicated in the header line of the class declaration. This process is making use of the object oriented concept of inheritance, which we explore in more detail later.

JApplet classes must also be declared as public.

The paint method is one of several applet methods that have particular significance. It is invoked automatically whenever the graphic elements of the applet need to be painted to the screen, such as when the applet is first run or when another window that was covering it is moved.

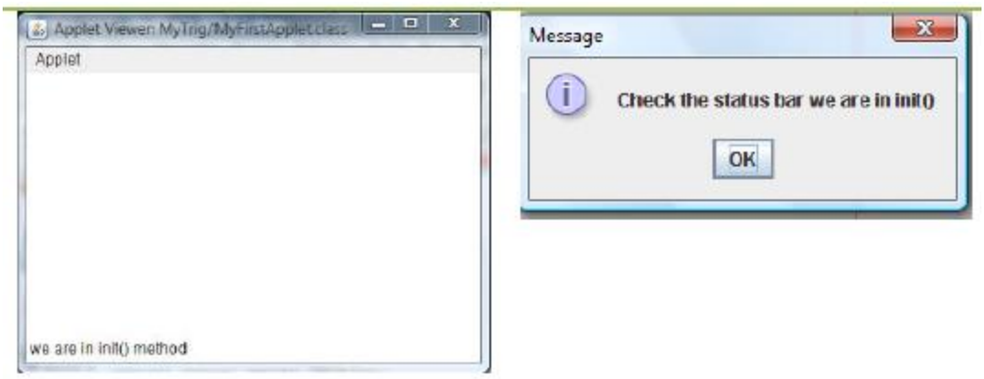
We will create a JApplet class – MyFirstApplet.java – to demonstrate how a simple applet can be created and the life-cycle of an applet. The complete code listing is presented below.

```
/**
 *
 * @author Mr. HySimon
 */
 import java.awt.Graphics;
import javax.swing.JOptionPane;
import javax.swing.JApplet;
public class NewJApplet extends JApplet {

public void init()
{
showStatus( "we are in init() method " );
JOptionPane.showMessageDialog( null,
"Check the status bar we are in init() " );
} // end method init
public void start()
{
showStatus( "we are in start() method ");
JOptionPane.showMessageDialog( null,
"Check the status bar we are in start() " );
} // end method start
public void paint(Graphics g)

 {
super.paint( g );
showStatus( "we are in paint() method ");
JOptionPane.showMessageDialog( null,
"Check the status bar we are in paint() " );
} // end method paint
public void stop()
{
showStatus( "we are in stop() method ");
JOptionPane.showMessageDialog( null,
"Check the status bar we are in stop() " );
} // end method stop
public void destroy()
{
showStatus( "we are in destroy() method ");
JOptionPane.showMessageDialog( null,
"Check the status bar we are in destroy() " );
} // end method destroy
} // end JApplet class
```

Output



Applet Life-Cycle Methods

Now that you have created an applet, let's consider the five applet methods that are called by the applet container from the time the applet is loaded into the browser to the time that the applet is terminated by the browser. These methods correspond to various aspects of an applet's life cycle.

The table below lists these methods, which are inherited into your applet classes from class JApplet. The table specifies when each method gets called and explains its purpose. Other than method paint, these methods have empty bodies by default. If you would like to declare any of these methods in your applets and have the applet container call them, you must use the method headers shown in the table. If you modify the method headers (e.g., by changing the method names or by providing additional parameters), the applet container will not call your methods.

Instead, it will call the superclass methods inherited from JApplet.

Method: When the method is called and its purpose

public void init()

Called once by the applet container when an applet is loaded for execution. This method initializes an applet. Typical actions performed here are initializing fields, creating GUI components, loading sounds to play, loading images to display etc.

public void start()

Called by the applet container after method init completes execution. In addition, if the user browses to another Web site and later returns to the applet's HTML page, method start is called again. The method performs any tasks that must be completed when the applet is loaded for the first time and that must be performed every time the applet's HTML page is revisited. Actions performed here might include starting an animation

public void paint(Graphics g)

Called by the applet container after methods init and start. Method paint is also called when the applet needs to be repainted. For example, if the user covers the applet with another open window on the screen and later uncovers the applet, the paint method is called. Typical actions performed here involve drawing with the Graphics object g that is passed to the paint method by the applet container.

public void stop()

This method is called by the applet container when the user leaves the applet's Web page by browsing to another Web page. Since it is possible that the user might return to the Web page containing the applet, method stop performs tasks that might be required to suspend the applet's execution, so that the applet does not use computer processing time when it is not displayed on the screen. Typical actions performed here would stop the execution of animations and threads.

public void destroy()

This method is called by the applet container when the applet is being removed from memory. This occurs when the user exits the browsing session by closing all the browser windows and may also occur at the browser's discretion when the user has browsed to other Web pages. The method performs any tasks that are required to clean up resources allocated to the applet.

Module 6:

Algorithms

Any computing problem can be solved by executing a series of actions in a specific order. A procedure for solving a problem in terms of the **actions** to execute and the order in which these actions execute is called an **algorithm**.

Pseudocode

Pseudocode is an informal language that helps programmers develop algorithms without having to worry about the strict details of Java language syntax. The pseudocode we present is particularly useful for developing algorithms that will be converted to structured portions of Java programs. Pseudocode is similar to everyday English it is convenient and user friendly, but it is not an actual computer programming language.

Sequence Structure in Java

The sequence structure is built into Java. Unless directed otherwise, the computer executes Java statements one after the other in the order in which they are written, that is, in sequence. The diagram below illustrates a typical sequence structure in which two calculations are performed in order. Java lets us have as many actions as we want in a sequence structure. As we will soon see, anywhere a single action may be placed, we may place several actions in sequence.

Selection Statements in Java

Java has three types of selection statements.

1. **The `if` statement** either performs (selects) an action if a condition is true or skips the action, if the condition is false. The if statement is a single-selection statement because it selects or ignores a single action (or, as we will soon see, a single group of actions).
2. **The `if...else` statement** performs an action if a condition is true and performs a different action if the condition is false. The `if...else` statement is called a double selection statement because it selects between two different actions (or groups of actions).
3. **The `switch` statement** performs one of many different actions, depending on the value of an expression. The `switch` statement is called a multiple-selection statement because it selects among many different actions (or groups of actions).

if .... Single-Selection Statement

Programs use selection statements to choose among alternative courses of action. For example, suppose that the passing grade on an exam is 40. The pseudocode statements is:

1. Begin
2. Input student marks
3. If marks is greater than or equal to 40
4. Print "Passed"
5. End

determines whether the condition "student's grade is greater than or equal to 40" is true or false. If the condition is true, "Passed" is printed.

Program code:

```
public class StudentGrade {
  public static void main(String[] args) {
        String grade;
         double marks;
           marks = 45;
        if (marks >= 40)
          grade = "Passed";
          System.out.println("Student grade is: " + grade);

  }
}
```

`if...else` Double-Selection Statement

The `if` single-selection statement performs an indicated action only when the condition is **`true`**; otherwise, the action is skipped. The `if...else` double-selection statement allows the programmer to specify an action to perform when the condition is true and a different action when the condition is false. For example, the pseudocode statement:

1. Begin
2. Input student marks
3. If marks is greater than or equal to 40
4. Print "Passed"
5. Else
6. Print "Failed"
7. End

prints "Passed" if the student's grade is greater than or equal to 50, but prints "Failed" if it is less than 50. In either case, after printing occurs.

Program code:

```
public class StudentGrade {
   public static void main(String[] args) {
      String grade;
      double marks;
      marks = 45;
      if (marks >= 40)
          grade = "Passed";
      else
          grade = "Failed";
          System.out.println("Student grade is: " + grade);
     }
}
```

Note that the body of the else is also indented. Whatever indentation convention you choose should be applied consistently throughout your programs. It is difficult to read programs that do not obey uniform spacing conventions.

Conditional Operator (?:)

Java provides the conditional operator (?:) that can be used in place of an if...else statement.
This is Java's only ternary operator this means that it takes three operands. Together, the operands and the ?: symbol form a conditional expression. The first operand (to the left of the ?) is a boolean expression (i.e., a condition that evaluates to a boolean valuetrue or false), the second operand (between the ? and :) is the value of the conditional expression if the Boolean expression is TRue and the third operand (to the right of the :) is the value of the conditional expression if the boolean expression evaluates to false. For example, the statement

```
System.out.println( studentGrade >= 40 ? "Passed" : "Failed" );
```

prints the value of println's conditional-expression argument. The conditional expression in this statement evaluates to the string "Passed" if the boolean expression studentGrade >= 50 is true and evaluates to the string "Failed" if the boolean expression is false. Thus, this statement with the conditional operator performs essentially the same function as the if...else statement shown earlier in this section. The precedence of the conditional operator is low, so the entire conditional expression is normally placed in parentheses.

Nested `if...else` Statements

A program can test multiple cases by placing `if...else` statements inside other `if...else` statements to create **`nested if...else statements`**. For example, the following pseudocode represents a nested `if...else` that prints `A` for exam marks greater than or equal to 80, `B` for grades in the range 60 to 79, `C` for grades in the range 46 to 59, `D` for grades in the range 40 to 45 and `F` for all other grades:
Presudocode for if….else statements

1. Begin
2. Input student marks
3. If marks is greater than or equal to 80
4. Print "A"
5. else if marks is greater than or equal to 60
6. Print "B"

7. Else
8. If marks is greater than or equal to 46
9. Print "C"
10. Else
11. If marks is greater than or equal to 40
12. Print "D"
13. Else
14. Print "F"

Blocks

The `if` statement normally expects only one statement in its body. To include several statements in the body of an `if` (or the body of an `else` for an `if...else` statement), enclose the statements in braces (`{` and `}`). A set of statements contained within a pair of braces is called a **block**. A block can be placed anywhere in a program that a single statement can be placed.

The following example includes a block in the `else`-part of an `if...else` statement:

```
if ( grade >= 40 )
      System.out.println( "Passed" );
else
{
      System.out.println( "Failed" );
      System.out.println( "You must take this course again." );
}
```

In this case, if `grade` is less than 40, the program executes both statements in the body of the `else` and prints

```
Failed.
You must take this course again.
```

Note the braces surrounding the two statements in the `else` clause. These braces are important. Without the braces, the statement

```
System.out.println( "You must take this course again." );
```

would be outside the body of the `else`-part of the `if...else` statement and would execute regardless of whether the grade was less than 40.

Types of errors that may exist when using block:
Syntax errors: when one brace in a block is left out of the program. They are caught by the compiler.
Logic error: when both braces in a block are left out of the program. It has its effect at execution time.
Fatal logic error: causes a program to fail and terminate prematurely.
*Nonfatal logic error***:** allows a program to continue executing, but causes the program to produce incorrect results.

Module 7:

The `while` Repetition Statement

A **repetition statement** (also called a looping statement or a **loop**) allows the programmer to specify that a program should repeat an action while some condition remains true.
The statement(s) contained in the While repetition statement constitute the body of the While repetition statement, which may be a single statement or a block. Eventually, the condition will become false. At this point, the repetition terminates, and the first statement after the repetition statement executes.

For example :
consider a program designed to find the first power of 3 larger than 100. Suppose that the `int` variable `product` is initialized to `3`. The following `while` statement finishes executing, `product` contains the result.

Program Code:

```
public class WhileTest {
   public static void main(String[] args) {
        int product = 3;
        while (product <= 100)
              product = 3 * product;
              System.out.println("Product: " + product);
    }
}
```

When this `while` statement begins execution, the value of variable `product` is 3. Each iteration of the `while` statement multiplies `product` by 3, so `product` takes on the values 9, 27, 81 and 243 successively. When variable `product` becomes 243, the `while` statement condition `product <= 100`becomes false. This terminates the repetition, so the final value of `product` is 243. At this point, program execution continues with the next statement after the `while` statement.

Formulating Algorithms: Counter-Controlled Repetition

To illustrate how algorithms are developed, we will create a class `TenNos` (declared in TenNos.java) to generate and sum the first ten integer numbers from 1 to 10 by default. Below is the algorithm (pseudocode for generating and calculating the sum of ten numbers from 1 to 10.

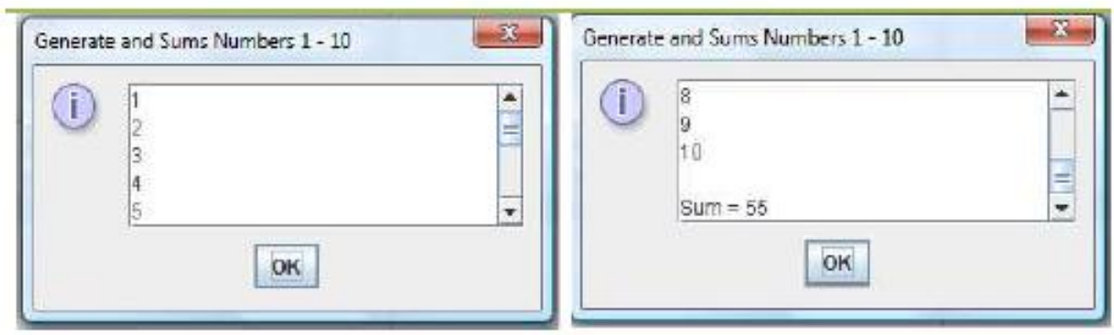
Pseudocode: To generate an calculate the sum of the first ten numbers from 1 to 10
First Pseudocode:
Step1: Initialize variables (counter = 1, sum = 0, n = 0)
Step2: Generate numbers from 1 to 10 (While counter is less than or equal to n)
Step3: Calculate sum of the numbers (add counter to sum)
Step4: Increment counter by 1
Step5: Return to step 2
Step6: Display numbers and sum
Step7: Stop.

Based on this algorithm the Java program in declared as TenNos.java was developed. The code listing is presented below:

```
1 /*
2 * TenNos.Java
3 * Generates the First Ten Numbers and Calculate their Sum
4 *
5 */
6
7
8 import javax.swing.JOptionPane;
9 import javax.swing.JTextArea;
10 import javax.swing.JScrollPane;
11
12 public class TenNos
13 {
14
15 public static void main(String[] args)
16 {
17 int sum = 0;
18 int counter = 1;
19 int n = 10;
20 String nos = "";
21
22 // Create JTextArea for displaying numbers
23 JTextArea output = new JTextArea( 5, 20 );
24
25 // Generate numbersn
```

```
26 while(counter <= n ){
27 output.append(counter + "\n"); // add numbers to the JTextarea
28 sum += counter; // calculate sum
29 counter++; // increment counter by 1
30 } // end while i <= n
31
32 nos = "\nSum = " + sum;
33 output.append(nos);
34
35 // Append a JScrollpane to the JTextArea object
36 JScrollPane outputArea = new JScrollPane( output );
37
38 // Display numbers and their sum
39 JOptionPane.showMessageDialog(null, outputArea,
40 "Generate and Sums Numbers 1 - 10",
41 JOptionPane.INFORMATION_MESSAGE);
42
43 } // end method main
44
45 } // end of class TenNos
```

Output:



Module 8:

Fundamentals of Characters and Strings

Characters are the fundamental building blocks of Java source programs. Every program is composed of a sequence of characters that when grouped together meaningfully are interpreted by the computer as a series of instructions used to accomplish a task. A program may contain **character literals**. A character literal is an integer value represented as a character in single quotes. For example, `'z'` represents the integer value of `z`, and `'\n'` represents the integer value of newline. The value of a character literal is the integer value of the character in the **Unicode character set**.

What are Strings?

A string is a sequence of characters treated as a single unit. A string may include letters, digits and various **special characters**, such as `+`, `-`, `*`, `/` and `$`. A string is an object of class `String`.

String literals (stored in memory as `String` objects) are written as a sequence of characters in double quotation marks, as in:

`"John Q. Doe"` (a name)

`"9999 Main Street"` (a street address)

`"Waltham, Massachusetts"` (a city and state)

`"(201) 555-1212"` (a telephone number)

A string may be assigned to a `String` reference. The declaration

```
String color = "blue";
```

initializes `String` reference color to refer to a String object that contains the string `"blue"`.

Arrays

An array is a group of variables (called **elements** or **components**) containing values that all have the same type. Recall that types are divided into two categories primitive types and reference types. Arrays are objects, so they are considered reference types. As you will soon see, what we typically think of as an array is actually a reference to an array object in memory. The elements of an array can be either primitive types or reference types. To refer to a particular element in an array, we specify the name of the reference to the array and the position number of the element in the array. The position number of the element is called the element's **index** or **subscript.**

Declaring and Creating Arrays

Array objects occupy space in memory. Like other objects, arrays are created with keyword `new`. To create an array object, the programmer specifies the type of the array elements and the number of elements as part of an **array-creation expression** that uses keyword `new`. Such an expression returns a reference that can be stored in an array variable. The following declaration and array-creation expression create an array object containing 12 `int` elements and store the array's reference in variable `c`:

```
int c[] = new int[ 12 ];
```

Examples Using Arrays

This section presents several examples that demonstrate declaring arrays, creating arrays, initializing arrays and manipulating array elements.

Creating and Initializing an Array

The program uses keyword `new` to create an array of 10 `int` elements, which are initially zero (the default for `int` variables).

The following program Initializing the elements of an array to default values of zero.

```
1 // arrays
2 // Creating an array.
3
4 public class InitArray
5 {
6 public static void main( String args[] )
7 {
8 int array[]; // declare array named array
9
10 array = new int[ 10 ]; // create the space for array
11
12 System.out.printf( "%s%8s\n", "Index", "Value" ); //column headings
13
14 // output each array element's value
15 for ( int counter = 0; counter < array.length; counter++ )
16 System.out.printf( "%5d%8d\n", counter, array[ counter ] );
17 } // end main
18 } // end class InitArray
```

Output

```
Index   Value
    0       0
    1       0
    2       0
    3       0
    4       0
    5       0
    6       0
    7       0
    8       0
    9       0
```

Module 9:

A **graphical user interface (GUI)** presents a user-friendly mechanism for interacting with an application. A GUI gives an application a distinctive "look" and "feel." Providing different applications with consistent, intuitive user interface components allows users to be somewhat familiar with an application, so that they can learn it more quickly and use it more productively.

GUIs are built from **GUI components**. These are sometimes called controls or widgets short for window gadgets in other languages. A GUI component is an object with which the user interacts via the mouse, the keyboard or another form of input, such as voice recognition.

Overview of Swing Components

Though it is possible to perform input and output using the `JOptionPane` dialogs presented in earlier weeks, most GUI applications require more elaborate, customized user interfaces. The remainder of this text discusses many GUI components that enable application developers to create robust GUIs.

The following table lists several **Swing GUI components** from package **`javax.swing`** that are used to build Java GUIs.

Component	Description
`JLabel`	Displays uneditable text or icons.
`JTextField`	Enables user to enter data from the keyboard. Can also be used to display editable or uneditable text.
`JButton`	Triggers an event when clicked with the mouse.
`JCheckBox`	Specifies an option that can be selected or not selected.
`JComboBox`	Provides a drop-down list of items from which the user can make a selection by clicking an item or possibly by typing into the box.
`JList`	Provides a list of items from which the user can make a selection by clicking on any item in the list. Multiple elements can be selected.
`JPanel`	Provides an area in which components can be placed and organized. Can also be used as a drawing area for graphics.

Displaying Text and Images in a Window

Our next example introduces a framework for building GUI applications. This framework uses several concepts that you will see in many of our GUI applications. This is our first example in which the application appears in its own window. Most windows you will create are an instance of class `JFrame` or a subclass of `JFrame`. `JFrame` provides the basic attributes and behaviors of a window, a title bar at the top of the window, and buttons to minimize, maximize and close the window. Since an application's GUI is typically specific to the application, most of our examples will consist of two classesa subclass of `JFrame` that helps us demonstrate new GUI concepts and an application class in which `main` creates and displays the application's primary window.

The application of GUI shown below demonstrates several `JLabel` features and presents the framework we use in most of our GUI example.